**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**
**Department of Computer Science and Engineering**
**ASSESSMENT TOOL 1 – RAG-BASED MINI ASSIGNMENT (DOCUMENT QA / AI AGENT DEMO)**

| | | | |
|---|---|---|---|
| **Course Code:** | CSA65 | **Course Title:** | Generative AI and Large Language Models |
| **CO Assessed:** | CO3 – Precision (BL2, BL3) | **Assessment:** | Assessment Tool 2 – RAG-based Mini Assignment (Document QA / AI Agent Demo) |
| **Weightage:** | 20% of CIA | **Total Marks:** | 2 * 50 = 100 (1 Question1) |
| **CO3 Statement:** | *Develop intelligent applications using Retrieval-Augmented Generation (RAG), vector databases, and introductory AI agent concepts.(BL3, BL4)* | | |
| **Student Name:** | | **Reg. No.:** | |
| **Section / Batch:** | | **Date:** | |

**Instructions to Students**

- Answer 2 questions. Each question carries 50 marks.Total: 100 Marks.

· Implement the solution using **Python**.
· Use an appropriate LLM such as **OpenAI, Gemini, or Hugging Face**.
· Use **embeddings and semantic search** where applicable.
· Use **FAISS or ChromaDB** as the vector database for RAG tasks.
· Demonstrate the complete pipeline:
**Document → Chunking → Embedding → Retrieval → Generation**
· Demonstrate the system with multiple queries.
· Handle irrelevant, incomplete, or unsupported queries appropriately.
· Display retrieved context/source wherever applicable.
· Explain the system architecture.
· Provide a working end-to-end demonstration.

**Code of Conduct**

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

9. **Semantic Search over Multiple Documents**: Create at least 10 documents and implement semantic search using embeddings. Retrieve the top three most relevant documents for a query.

## Q1. Semantic Search over Multiple Documents

### 1. Aim

To develop a semantic search system using embeddings and FAISS that can search through multiple documents and retrieve the top three most relevant documents for a given user query.

### 2. Objective

The objective of this implementation is to understand how semantic search works using text embeddings and a vector database. Unlike traditional keyword-based search, semantic search identifies documents based on the meaning of the query. In this system, 10 documents related to Artificial Intelligence are converted into embeddings and stored in a FAISS vector database. When the user enters a query, the system finds and displays the three most relevant documents.

### 3. Technologies Used

- Python
- Google Colab
- Sentence Transformers
- all-MiniLM-L6-v2 embedding model
- FAISS
- NumPy

### 4. Documents Used

The system contains the following 10 documents:

1. Artificial Intelligence
2. Machine Learning
3. Deep Learning
4. Natural Language Processing
5. Computer Vision
6. Generative AI
7. Large Language Models
8. Retrieval-Augmented Generation
9. AI Agents
10. AI Ethics

### 5. Methodology

The semantic search system follows these steps:

**Document Collection → Embedding Generation → FAISS Vector Database → User Query → Query Embedding → Semantic Search → Top 3 Relevant Documents**

First, 10 documents related to Artificial Intelligence were created. The Sentence Transformer model all-MiniLM-L6-v2 was used to convert the documents into numerical embeddings. These embeddings represent the semantic meaning of the documents.

The generated embeddings were then stored in a FAISS vector database. When a user enters a query, the query is also converted into an embedding using the same model. FAISS compares the query embedding with the stored document embeddings and retrieves the three most relevant documents based on semantic similarity.

## 6. Implementation

The implementation was carried out using Python in Google Colab. Sentence Transformers was used for generating embeddings, while FAISS was used for efficient similarity search.

The system successfully generated embeddings for all 10 documents and stored them in the FAISS index. The user can enter different queries and the system retrieves the top three relevant documents along with their distance values and document content.

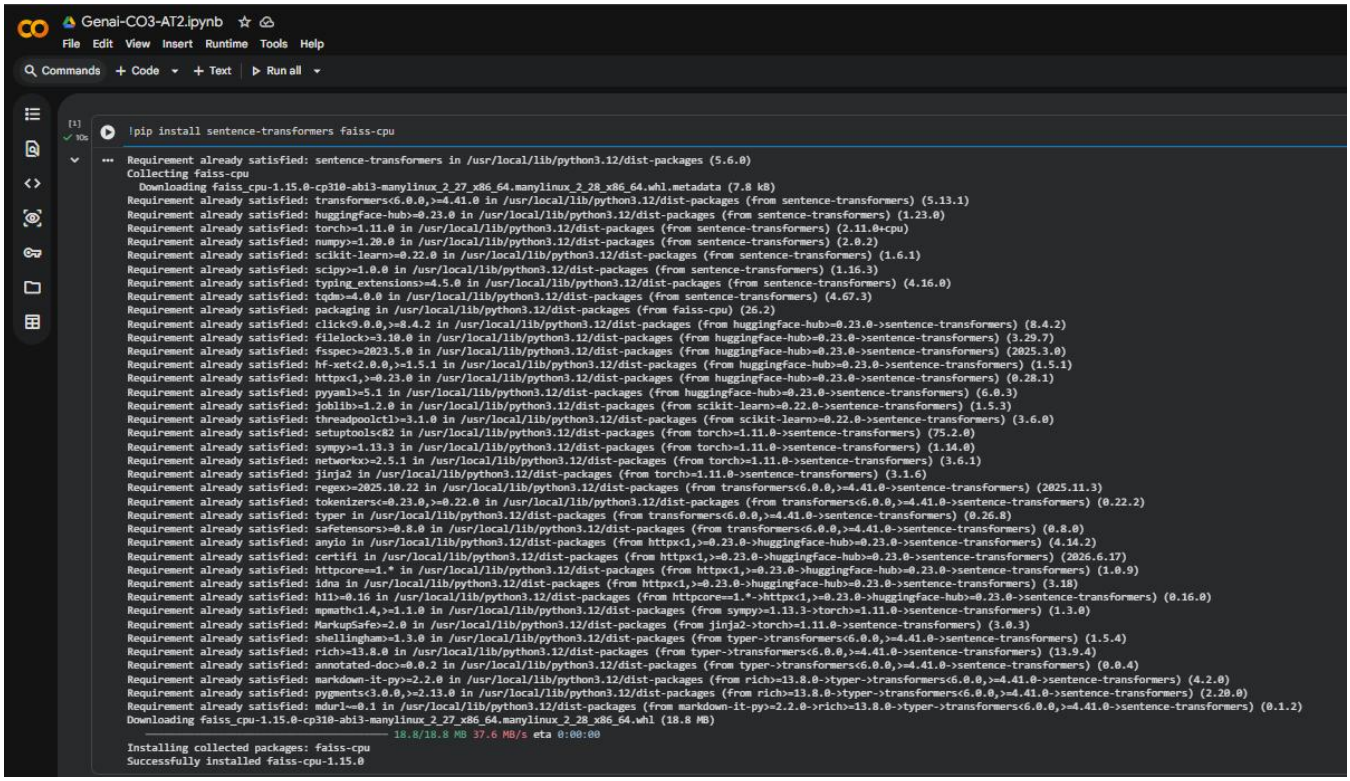



**Fig 1: Import Libraries**

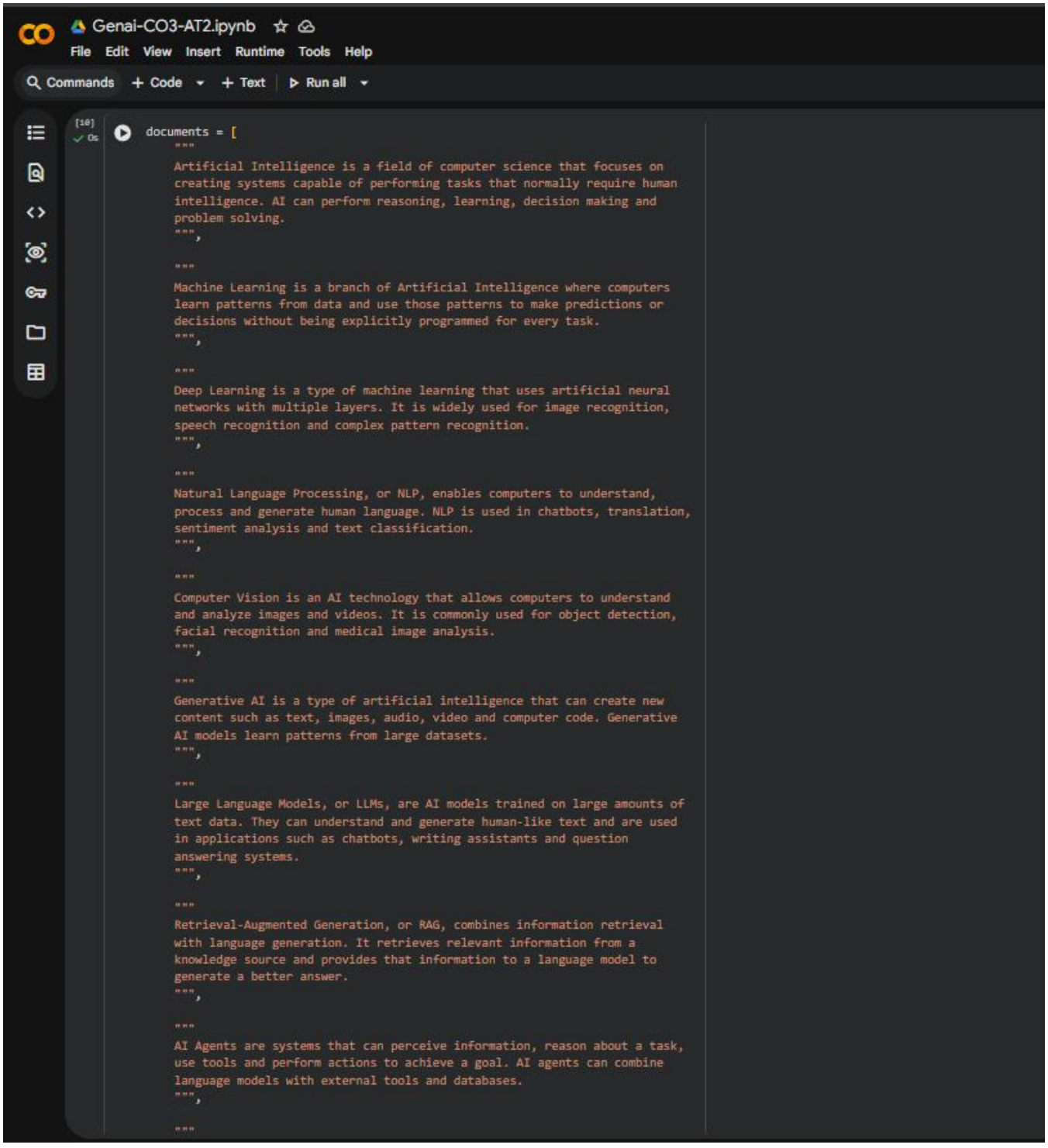



**Fig 2: Creating 10 documents**


**Fig 3: Embedding Generation**

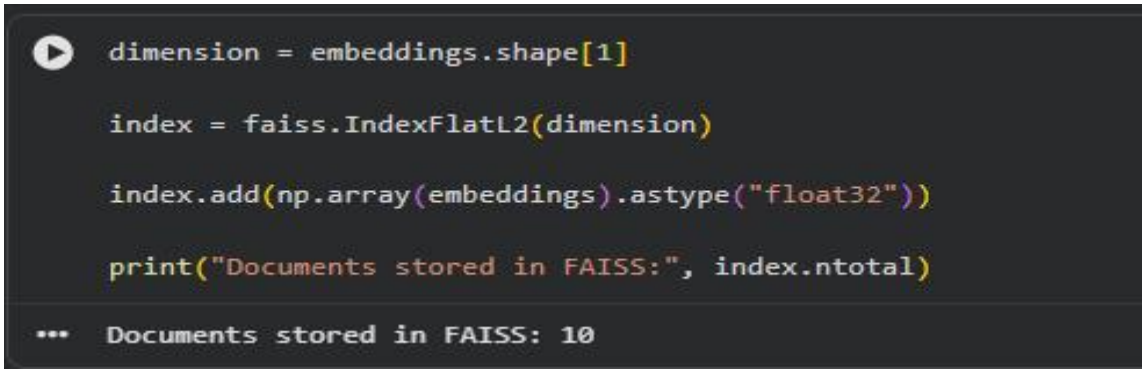



**Fig 4: FAISS database**

## 7. Sample Query and Output

### Query:

"What is Retrieval Augmented Generation?"

### Retrieved Results:

- Retrieval-Augmented Generation
- Generative AI
- Natural Language Processing

The system successfully identified the documents that were semantically related to the given query and displayed their content.

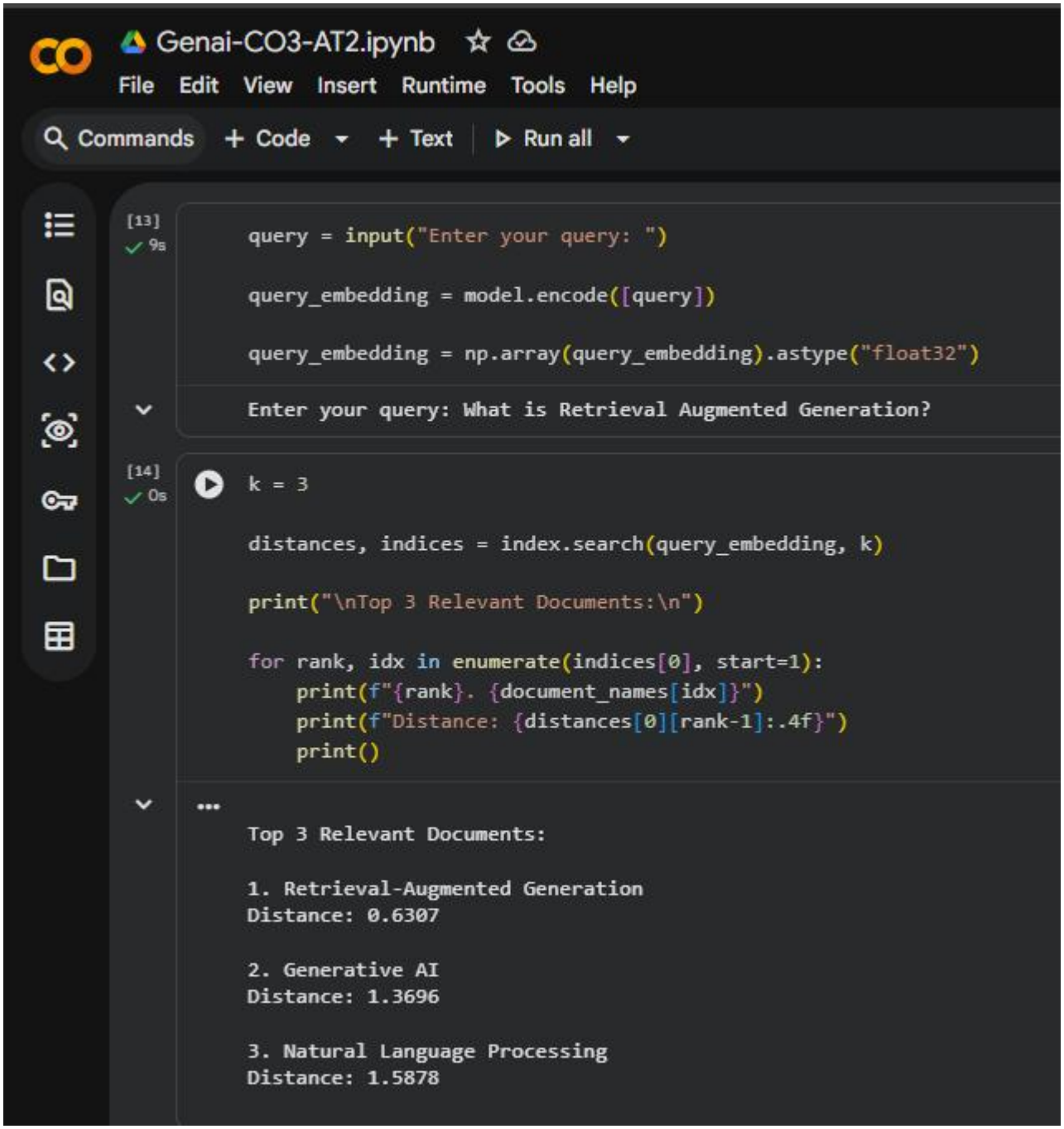



**Figure 5: Semantic Search Query and Top-3 Relevant Documents**

## 8. Multiple Query Testing

The system was tested with multiple queries to verify its semantic search capability. Example queries include:

- What is Retrieval Augmented Generation?
- What is machine learning?
- How do computers understand human language?
- What are the benefits of Generative AI?

For each query, the system retrieved the top three semantically relevant documents.

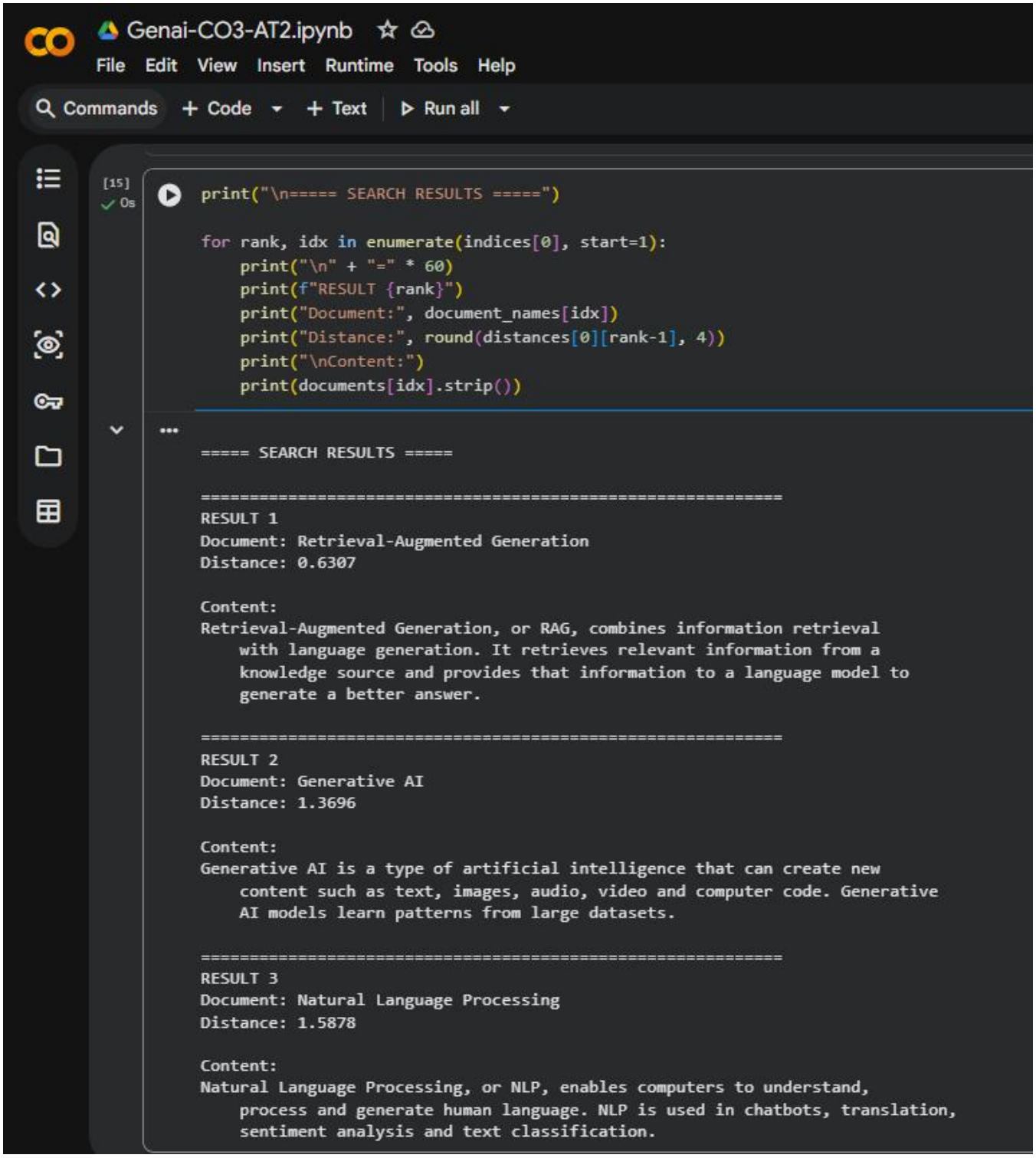



**Figure 7: Top-3 Retrieved Documents and Their Content**

## 9. Result

The semantic search system was successfully implemented using Sentence Transformer embeddings and FAISS. It was able to process user queries and retrieve the top three relevant documents based on semantic similarity.

## 10. Conclusion

This implementation demonstrates how embeddings and vector databases can be used to perform semantic search over multiple documents. The system can identify relevant information based on the meaning of a query rather than depending only on exact keyword matching. Thus, the required semantic search functionality was successfully achieved.

40. **Complete RAG + AI Agent Demonstration**: Develop a complete RAG-based AI Agent integrating document loading, chunking, embeddings, semantic search, FAISS/ChromaDB, retrieval, LLM generation, query handling, tool calling, source citation, and unsupported-query handling.

## Q2. Complete RAG + AI Agent Demonstration

## 1. Aim

To develop a complete Retrieval-Augmented Generation (RAG) based AI Agent that can load documents, divide them into chunks, generate embeddings, store them in a FAISS vector database, retrieve relevant information, generate answers using an LLM, provide source citations, and handle unsupported queries.

## 2. Objective

The objective of this implementation is to develop an intelligent document question-answering system using Retrieval-Augmented Generation. The system retrieves relevant information from a collection of documents before generating an answer. This helps the system provide answers based on the available documents instead of depending only on the language model's general knowledge.

The system also demonstrates semantic search, vector database usage, LLM-based answer generation, source citation, multiple query handling, and unsupported-query handling.

## 3. Technologies Used

- Python
- Google Colab
- Google Gemini 3.6 Flash
- Sentence Transformers
- all-MiniLM-L6-v2
- FAISS
- NumPy
- Google GenAI SDK

## 4. Documents Used

The RAG system uses 10 documents related to Artificial Intelligence:

- Artificial Intelligence
- Machine Learning
- Deep Learning
- Natural Language Processing
- Computer Vision
- Generative AI
- Large Language Models
- Retrieval-Augmented Generation
- AI Agents
- AI Ethics

These documents form the knowledge base of the RAG system.

## 5. System Architecture

The proposed system follows the architecture:

**Documents → Document Loading → Chunking → Embeddings → FAISS Vector Database → User Query → Semantic Retrieval → Relevant Context → Gemini LLM → Generated Answer → Source Citation**

The system first loads the documents and divides their content into smaller chunks. Each chunk is converted into a numerical embedding using the Sentence Transformer model. The embeddings are stored in a FAISS vector database.

When the user enters a question, the question is also converted into an embedding. FAISS performs a similarity search and retrieves the most relevant document chunks. These retrieved chunks are then provided as context to the Gemini language model. Gemini generates an answer based on the retrieved information. The system also displays the source documents used for generating the answer.

## 6. Document Loading

The documents were stored as individual text files in a rag_documents directory. Python was used to load the files and store their source names and contents.

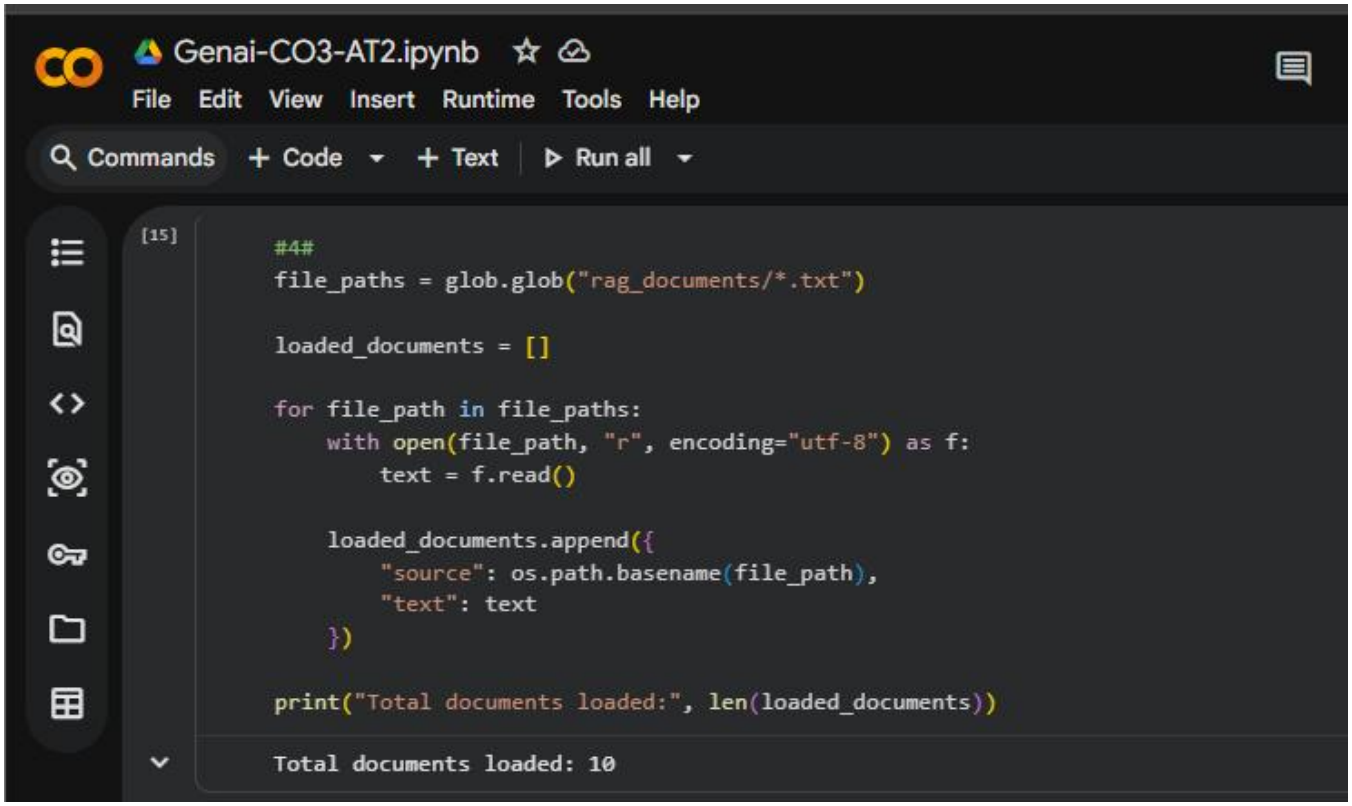



**Figure 1: Loading Multiple Documents for RAG**

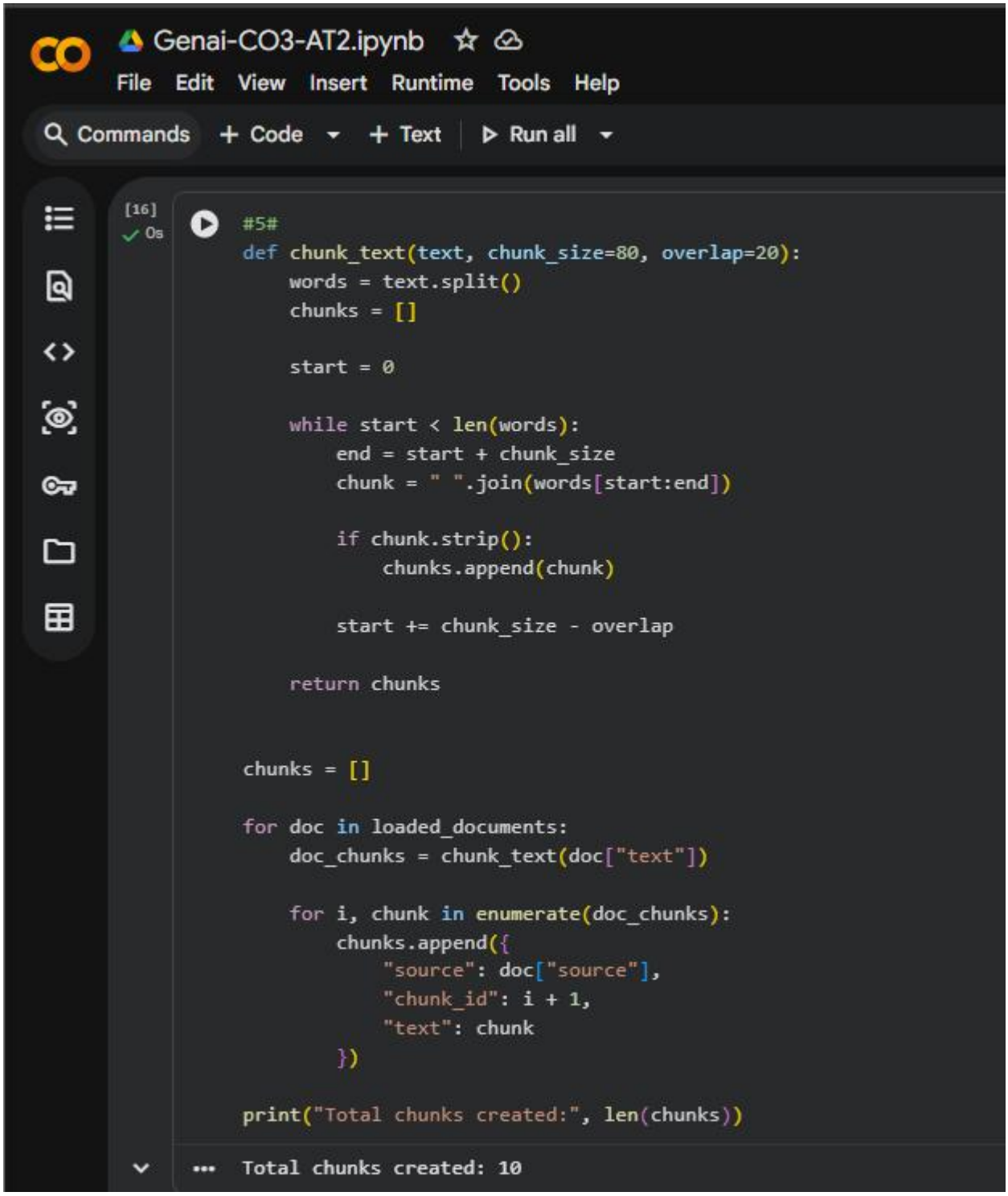
The system successfully loaded all 10 documents into the RAG pipeline.

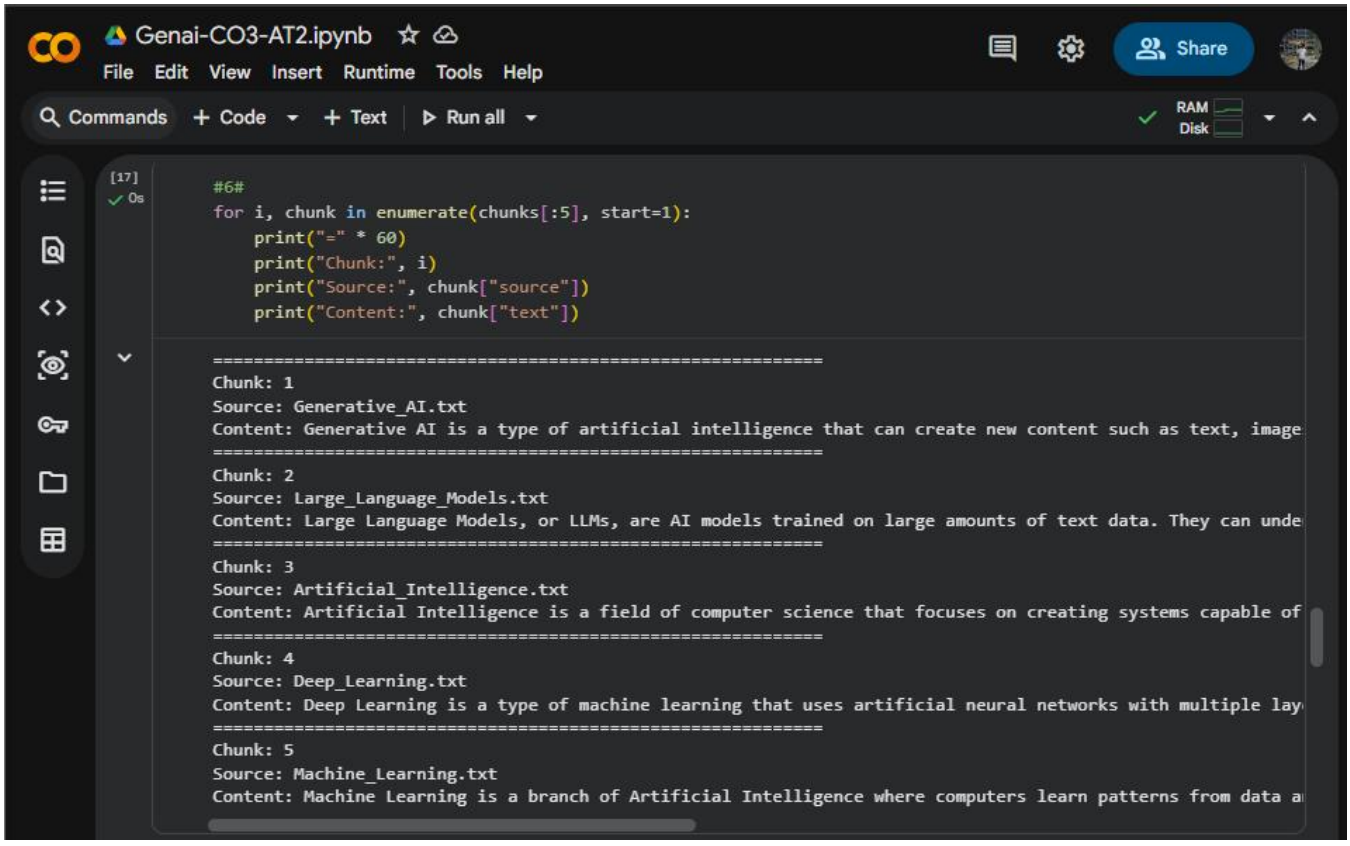
## 7. Document Chunking

After loading the documents, the text was divided into smaller chunks. Chunking helps the retrieval system search smaller and more meaningful sections of the documents.

An overlap was also used between consecutive chunks so that important information near the boundary of two chunks is not lost.

The chunks were stored along with their source document name and chunk number.


Genai-CO3-AT2.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
[16]
0s
#5#
def chunk_text(text, chunk_size=80, overlap=20):
words = text.split()
chunks = []
start = 0
while start < len(words):
end = start + chunk_size
chunk = " ".join(words[start:end])
if chunk.strip():
chunks.append(chunk)
start += chunk_size - overlap
return chunks
chunks = []
for doc in loaded_documents:
doc_chunks = chunk_text(doc["text"])
for i, chunk in enumerate(doc_chunks):
chunks.append({
"source": doc["source"],
"chunk_id": i + 1,
"text": chunk
})
print("Total chunks created:", len(chunks))
Total chunks created: 10




## 8. Embedding Generation

The Sentence Transformer model all-MiniLM-L6-v2 was used to generate embeddings for the document chunks.

An embedding represents the semantic meaning of text as a numerical vector. These vectors allow the system to compare the meaning of a user's query with the meaning of the stored document chunks.

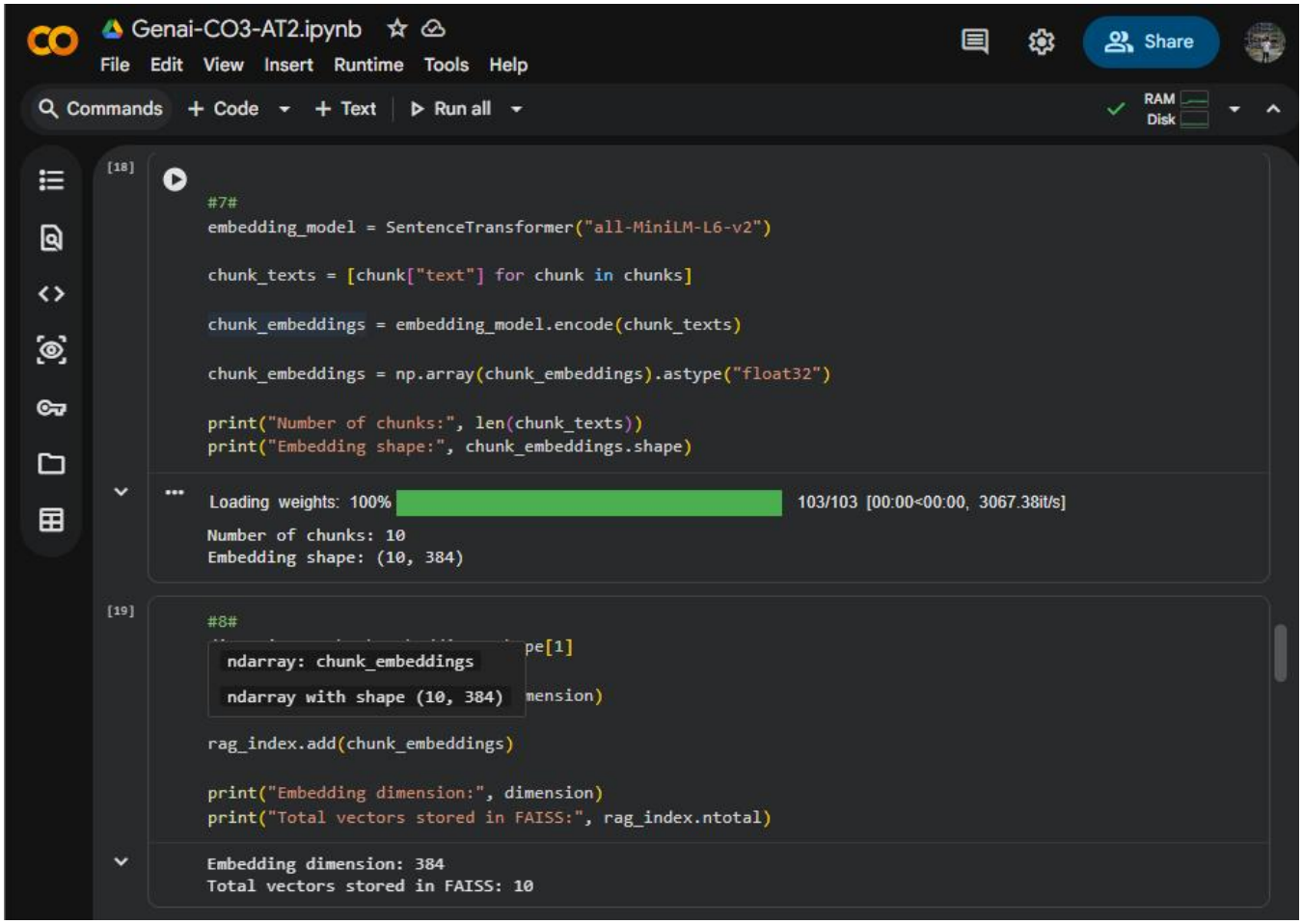
The generated embeddings were converted into numerical arrays and prepared for storage in the FAISS vector database.

## 9. FAISS Vector Database

FAISS was used as the vector database for storing the generated document embeddings.

The FAISS index performs similarity search between the query embedding and the stored document embeddings. The system retrieves the most relevant document chunks based on their similarity distance.

This allows the RAG system to perform efficient semantic retrieval.




**Embedding Generation for Document Chunks**

## 10. Semantic Retrieval

When a user enters a question, the question is converted into an embedding using the same Sentence Transformer model.

The query embedding is then compared with the embeddings stored in FAISS. The system retrieves the top relevant document chunks.

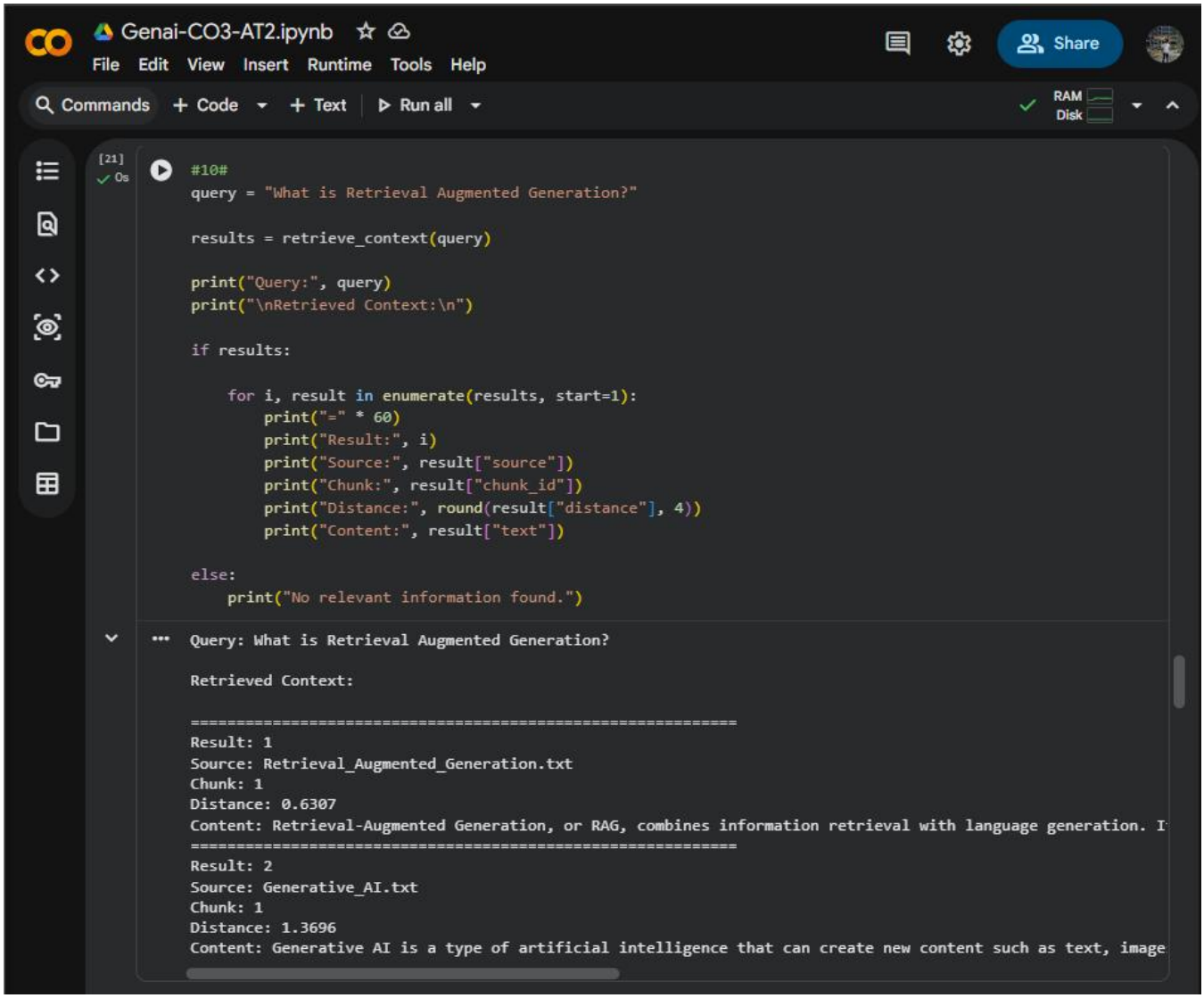
For example, for the query:

**"What is Retrieval Augmented Generation?"**

the system retrieved relevant information from documents such as:

- Retrieval-Augmented Generation
- Generative AI
- Large Language Models

The retrieved information is then used as context for generating the final answer.



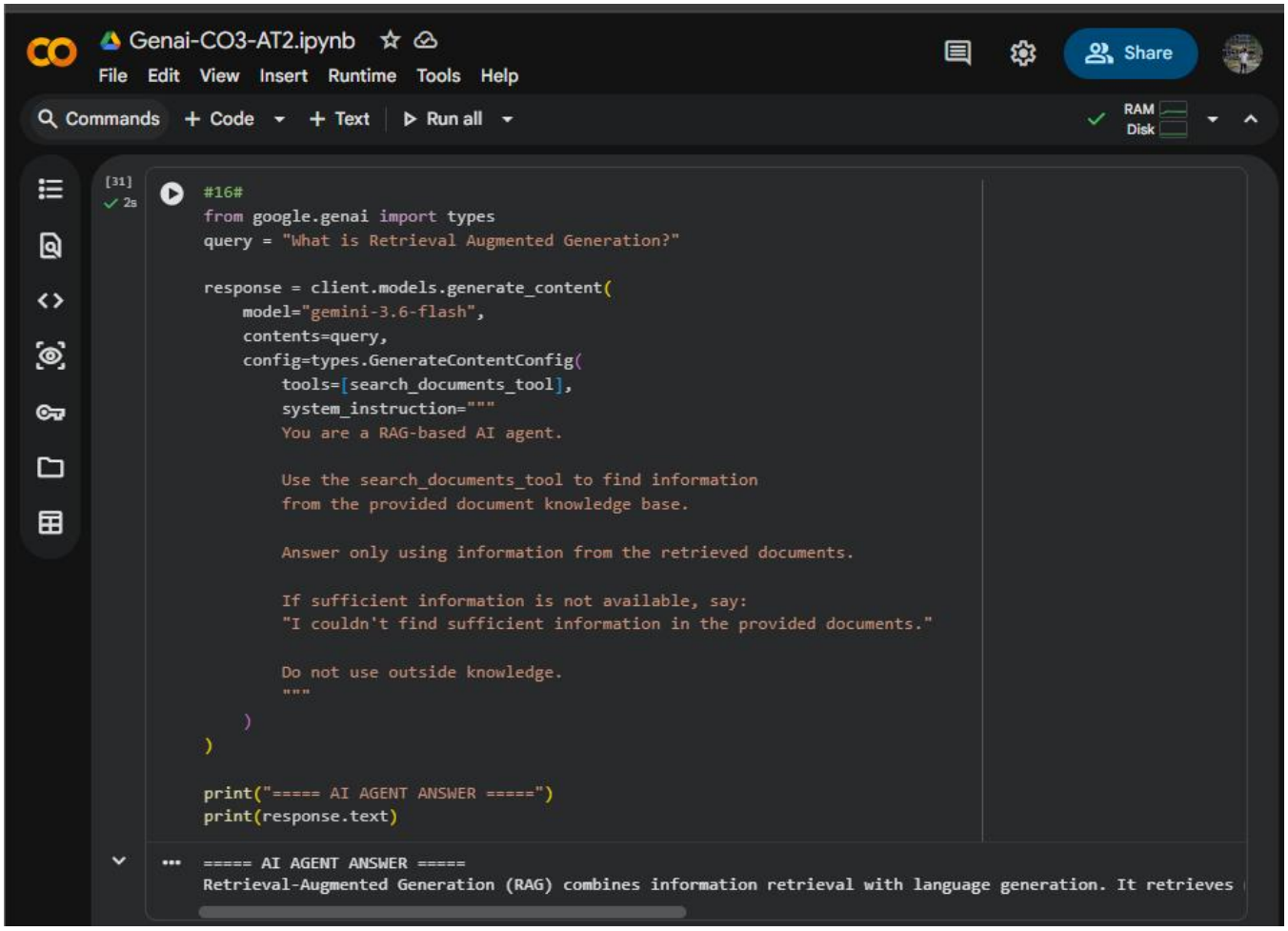

**Semantic Retrieval of Relevant Document Chunks**

## 11. Gemini LLM Integration

Google Gemini 3.6 Flash was integrated into the system as the Large Language Model.

The retrieved document information is provided to Gemini as context. The model generates an answer based on the retrieved information.

The system instruction also directs the model to answer using the provided document information and avoid using unsupported external information.




**Gemini-Based RAG Answer Generation**

## 12. RAG AI Agent and Tool Calling

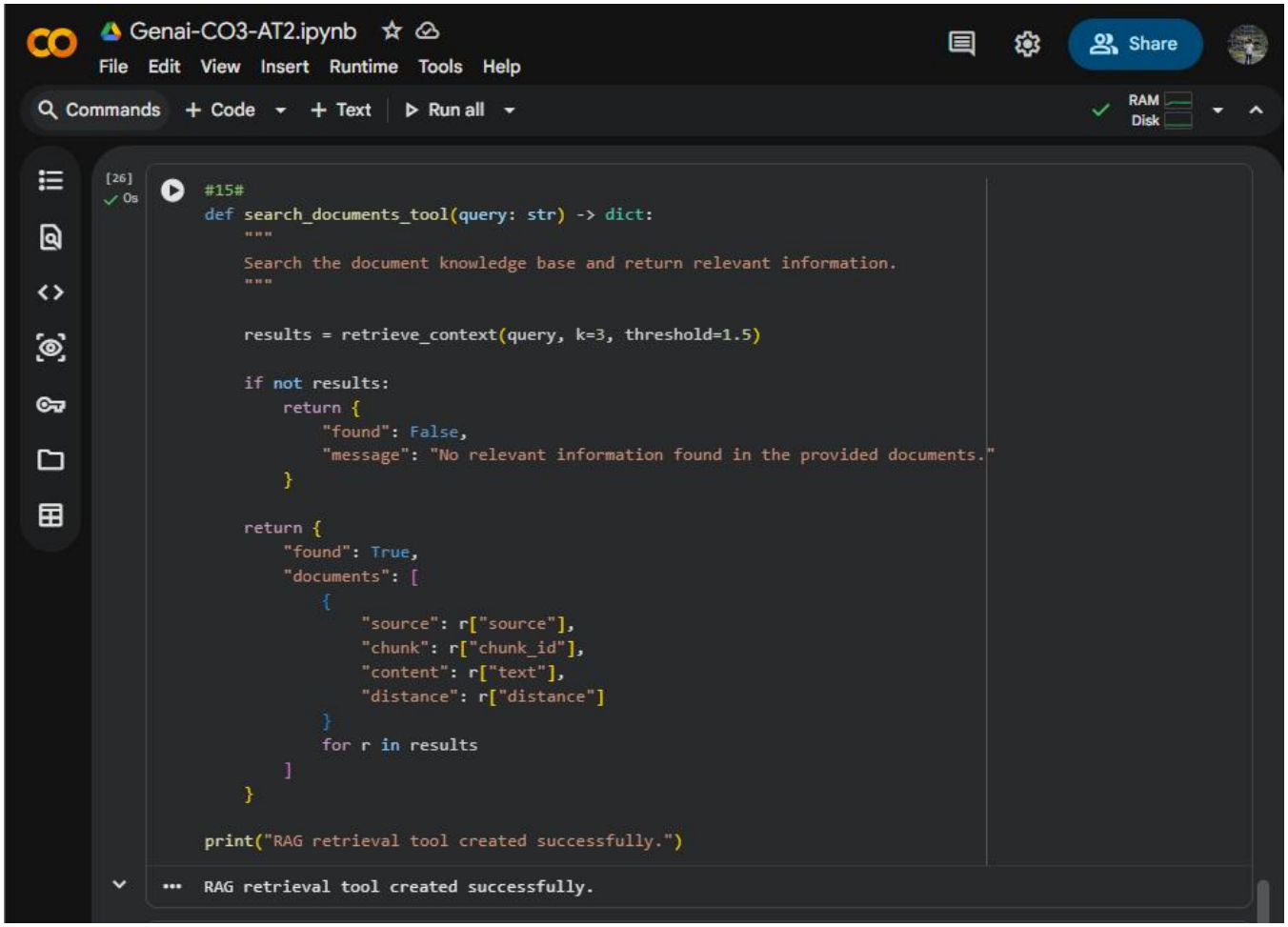
A document retrieval function was implemented as a tool for the AI Agent.

The tool searches the FAISS knowledge base whenever relevant information is required. The AI Agent can use this retrieval tool to obtain information from the documents before generating an answer.

The overall process is:

**User Query → AI Agent → Retrieval Tool → FAISS → Relevant Documents → Gemini → Final Answer**

This demonstrates the integration of retrieval and language generation in an AI Agent.




**RAG Retrieval Tool Used by the AI Agent**

## 13. Source Citation

The system displays the source documents used during retrieval along with the generated answer.

For example, when answering a question about Retrieval-Augmented Generation, the system displays sources such as:

- Retrieval_Augmented_Generation.txt
- Generative_AI.txt

The source information helps the user understand where the retrieved information came from and improves the transparency and groundedness of the generated answer.

## 14. Unsupported Query Handling

The system was also tested with queries that are not related to the information available in the document collection.

For unsupported questions, the system checks whether sufficiently relevant information is available in the knowledge base. If suitable information is not found, the system responds:

**"I couldn't find sufficient information in the provided documents."**

This prevents the system from providing unsupported answers when the required information is not present in the document collection.

## 15. Multiple Query Testing

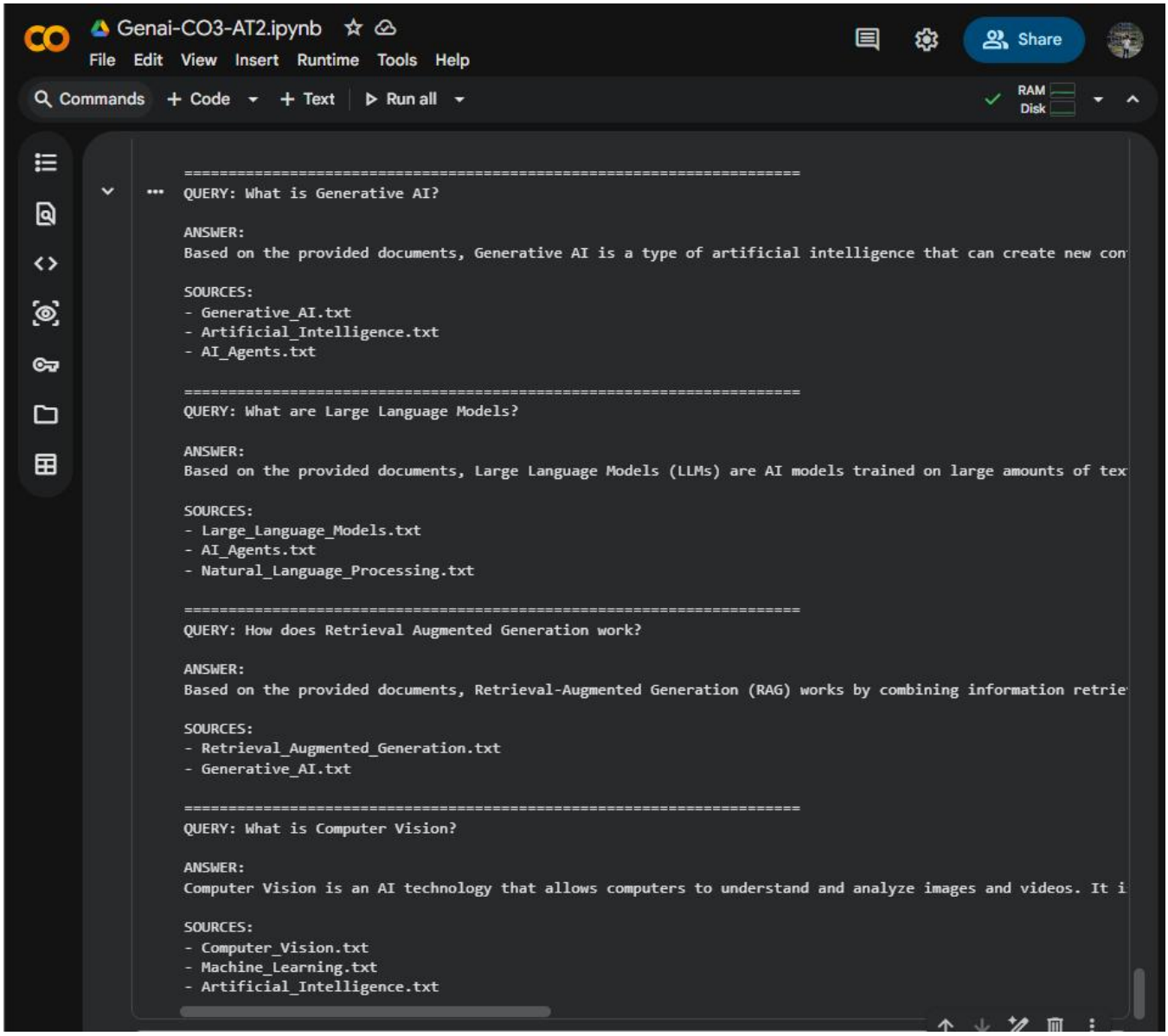
The RAG AI Agent was tested using multiple queries to verify its performance.

The following queries were tested:

- **What is Generative AI?**
- **What are Large Language Models?**
- **How does Retrieval Augmented Generation work?**
- **What is Computer Vision?**

For each query, the system retrieved relevant document information, generated an answer using Gemini, and displayed the source documents used.

The successful results demonstrate that the system can handle different queries over the same document knowledge base.




**Multiple Query Testing of the RAG AI Agent**

## 16. End-to-End Workflow

The complete workflow of the developed system is:

**Step 1:** Load the 10 documents.

**Step 2:** Divide the documents into smaller chunks.

**Step 3:** Generate embeddings using Sentence Transformer.

**Step 4:** Store the embeddings in FAISS.

**Step 5:** Accept a user query.

**Step 6:** Convert the query into an embedding.

**Step 7:** Perform semantic similarity search using FAISS.

**Step 8:** Retrieve the relevant document chunks.

**Step 9:** Provide the retrieved information to Gemini.

**Step 10:** Generate a grounded answer.

**Step 11:** Display the source documents.

**Step 12:** Handle unsupported queries appropriately.

## 17. Result

The complete RAG-based AI Agent was successfully implemented using Python, Sentence Transformers, FAISS, and Gemini 3.6 Flash.

The system successfully loads documents, performs chunking, generates embeddings, stores them in FAISS, retrieves relevant information, generates answers using the LLM, displays source documents, and handles multiple queries.

The implementation demonstrates a complete RAG pipeline from document processing to answer generation.

## 18. Conclusion

The RAG-based AI Agent successfully demonstrates how document retrieval and Large Language Models can be combined to build an intelligent question-answering system.

The use of embeddings and FAISS enables semantic retrieval of relevant information, while Gemini generates answers using the retrieved context. Source citation improves transparency, and unsupported-query handling reduces the possibility of generating answers when sufficient information is not available.

Therefore, the developed system demonstrates the complete Retrieval-Augmented Generation workflow along with introductory AI Agent functionality.

### RUBRICS FOR CALCULATION

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|---|---|---|---|---|---|
| **Pipeline Functionality** | 30% | Correct RAG pipeline — embeddings, retrieval and generation all function coherently | Pipeline mostly correct with minor gaps | Pipeline only partially functional | Pipeline non-functional or conceptually flawed |

| | | | | | |
|---|---|---|---|---|---|
| **Answer Relevance & Groundedness** | 25% | Retrieves relevant context and generates accurate, grounded answers | Mostly relevant/accurate answers | Inconsistent relevance or accuracy | Answers largely irrelevant or hallucinated |
| **Query Robustness** | 20% | Robust handling of varied queries and documents | Handles most queries well | Handles only simple queries | Fails on basic queries |
| **End-to-End Demo** | 15% | Smooth, well-demonstrated end-to-end system | Demo mostly smooth, minor hiccups | Demo has noticeable issues | Demo fails or is not ready |
| **Architecture Explanation** | 10% | Clear architecture diagram and explanation of design decisions | Adequate explanation of design | Minimal explanation of design | No explanation of design provided |

**Marks Evaluation:**

| Question No. | Pipeline Functionality(30%) | Answer Relevance & Groundedness (25%) | Query Robustness(20%) | End-to-End Demo(15%) | Architecture Explanation (10%) | Total Marks (100) |
|---|---|---|---|---|---|---|
| Q1 | | | | | | |
| Q2 | | | | | | |
| | | | | | **Overall Total (100)** | |